

Final Report: Steam Video Game Data Analysis

Team Name:

Team Lukas

Team members (DIS-04):

Lukas Leung, lukas777@stanford.edu

Henry Mao, henhen@stanford.edu

Rohan Swaminathan, rchamp@stanford.edu

Barton Li, bartonli@stanford.edu

1.0 Introduction:

For our final project, we wanted to explore whether machine learning could help answer questions about Steam games using real data. We used a cleaned dataset from Kaggle that included information such as price, genres, playtime, reviews, release date, and concurrent users. The idea was to apply predictive modeling techniques we learned in class, like Linear Regression, KNN, and GridSearchCV, to see how well we could predict different outcomes.

The questions we focused on covered a mix of classification and regression tasks (mainly regression). For example, we tried to predict whether a game would receive a positive or negative review based on its price, playtime, CCU, and ratings. We also looked at whether we could predict the year a game was released, estimate a game's concurrent user count, and predict things like the number of owners or total reviews based on game features. These questions were interesting to us because they combined useful business and user-facing insights, and they gave us a chance to work with multiple modeling tools in a hands-on way.

We chose these questions because they tied closely to what players and developers care about most success, popularity, and user engagement and they allowed us to experiment with multiple model types.

2.0 Dataset Description:

2.1 Data Provenance:

Dataset Name: *Steam Games Dataset 2025 (Cleaned)*

Source: Kaggle – Artermiloff ([Link](#))

Provenance: This dataset was downloaded from Kaggle and published by the user Artermiloff. It does not come with an academic publication, but it is a cleaned and enhanced version of the original Steam dataset, with estimated owner ranges and playtime data pre-processed for analysis.

Observational Unit: Each row represents a single video game listed on the Steam platform.

Key Features Used in this Project:

- release_date - The date the game was released to Steam
- price - Current price of the game
- genres - Genre or genres associated with the game
- positive - Number of positive reviews
- negative - Number of negative reviews
- num_reviews_total - Total numbers of reviews (positive + negative)
- estimated_owners - Range of ownership, converted to a numeric midpoint
- average_playtime_forever - Average total playtime per user
- average_playtime_2weeks - Average playtime in the past 2 weeks
- peak_ccu - Highest number of concurrent users recorded for the game

Size and Shape: The full dataset contains approximately 83,000 rows (games) and 30+ attributes. For our analysis, we focused on 10–12 features relevant to our research questions.

Missing Values: We dropped rows that had missing values in key fields like price, reviews, playtime, and peak CCU

2.2 Data Preparation:

Before prepping the data, we recognize the dataset has the following data quality issues:

- **Estimated Owners:** The original format was a string range. We split the range and calculated the numeric midpoint (average).
- **Release Date:** Parsed into a usable datetime format and transformed into numeric year or days since release where necessary.
- **Genres:** Games could have multiple genres; we used OneHotEncoder to convert this into dummy variables.
- **Outliers:** Extremely large values in playtime and ownership were not removed but were handled through feature scaling (e.g. StandardScaler) before training models.

To prepare the dataset for analysis, we started by selecting relevant columns from the original CSV file (game_march2025_cleaned.csv) provided in the Steam Games Dataset on Kaggle. This included attributes like genres, price, release_date, peak_ccu, positive, negative, estimated_owners, playtime, and platform availability (windows, mac, linux).

We processed the estimated_owners column, which was originally a string range, by converting it into a single numeric value using the average of the lower and upper bounds. We also converted Boolean platform columns (windows, mac, linux) to integers (0 or 1).

For numeric features such as price, positive, negative, and playtime, we applied standard scaling (StandardScaler) to ensure they were on a consistent scale for models sensitive to feature magnitude. For categorical features like genres, we used one-hot encoding.

In addition, we had to prepare data uniquely for specific questions, such as Release Date prediction, we converted the dates into numerical values (e.g., extracting the year) when it was needed. Other question-specific prep will be discussed further below.

We handled missing values by dropping rows with NaN values in key features that were required for modeling. Most preparation was done using pandas, scikit-learn preprocessing tools, and a few utility functions to clean and transform specific columns.

3.0 Exploratory Data Analysis:

Before beginning any modeling, we performed several exploratory data analysis (EDA) tasks to better understand the structure and relationships within the dataset and to guide our modeling decisions.

- **Distribution of Reviews:** We examined the distribution of positive & negative reviews, which helped us decide on a 70% threshold to define a game as having a “positive” or “negative” sentiment for classification tasks. This binary transformation allowed us to simplify the prediction problem while preserving meaningful differences between games.
- **Missing Data Check:** We inspected missing values across all columns and determined that some features, like price, playtime, and peak_ccu, had missing entries that needed to be dropped to preserve data integrity.
- **Handling of Extreme Values:** While some variables, such as playtime and concurrent users, exhibited long-tailed distributions, we chose not to remove extreme values to avoid losing information from popular or niche games.
- **Time Trends:** For questions involving release_date, we looked at the distribution of game release years to ensure there was enough variation across time to make it a viable prediction target. We also verified that the release_date field could be reliably converted to a numeric format for modeling.

4.0 Project Design, Implementation and Results:

Question 1:

Formulation of the Question: Can we predict whether a game will receive a positive or negative review based on its price, playtime, CCU, and ratings?

We defined a “positive review” as when the ratio of positive to total reviews (positive + negative) is at least 70%. This question is relevant because user sentiment is a key indicator of a game's success and is influenced by factors like pricing, gameplay length, and popularity. Accurately predicting review sentiment can help developers and marketers understand what drives positive reception.

Design:

We used the cleaned Steam dataset from Kaggle (games_march2025_cleaned.csv). To set up the target variable, we computed a review_score_binary column: 1 if the positive review ratio was ≥ 0.7 , and 0 otherwise. We also filtered out any games that had missing or invalid data, including those with zero reviews.

The features used for prediction included:

- price
- average_playtime_forever
- peak_ccu
- positive
- negative

We used two classification models: Logistic Regression and K-Nearest Neighbors (KNN) Classifier. The dataset was split into 80% training and 20% testing using train-test split. Model performance was evaluated using classification accuracy, supported by confusion matrices for both models.

Implementation:

We implemented classification pipelines using `make_pipeline()` for both models. For KNN, we applied `GridSearchCV` to tune two hyperparameters: `n_neighbors` and `metric` (euclidean, manhattan). The best model was selected using 5-fold cross-validation, optimizing for classification accuracy.

We chose accuracy because the goal was to correctly classify whether a review was positive or negative, and accuracy directly reflects the proportion of correct predictions, making it a straightforward and interpretable metric for binary classification tasks.

Results:

The Logistic Regression model achieved an accuracy of 68.69%, while the best KNN model (with manhattan distance and $k = 3$) achieved a much higher accuracy of 96.71%.

```
Logistic Regression
Accuracy: 0.6869013987459519
Confusion Matrix:
[[ 88 4543]
 [  1 9881]]
```

```
KNN Classifier
Best Params: {'kneighborsclassifier__metric': 'manhattan', 'kneighborsclassifier__n_neighbors': 3}
Accuracy: 0.9671329153173017
Confusion Matrix:
[[4392  239]
 [ 238 9644]]
```

This significant improvement shows that the KNN model was more effective at capturing complex, nonlinear relationships between the features and review sentiment. KNN leverages local similarity comparisons to make predictions, which likely helped it better understand patterns in user behavior metrics like playtime and CCU. Logistic Regression, with its linear assumptions, struggled to match this performance.

These results also suggest that user engagement metrics, such as playtime, price, and number of reviews, are strong predictors of overall game sentiment. A game with high playtime, affordable pricing, and a favorable review ratio is more likely to be positively reviewed, and KNN was better equipped to model this. (See Figure 1)

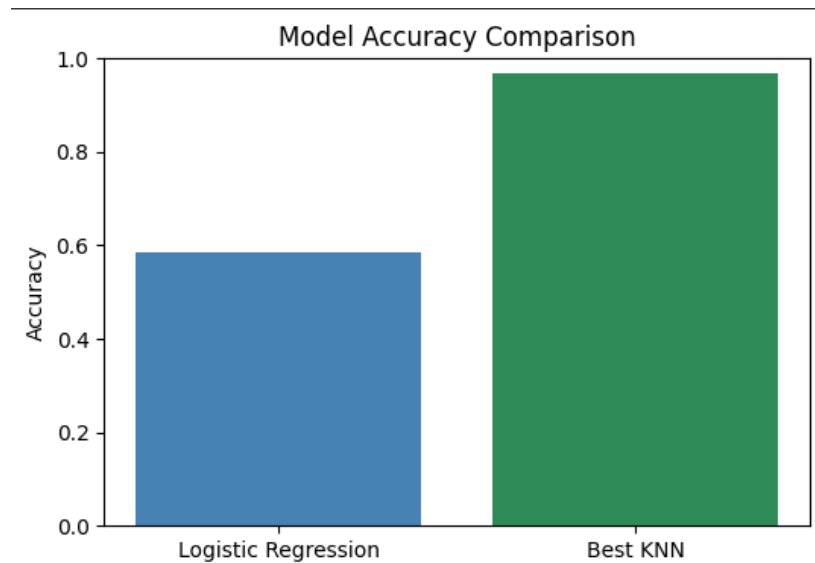


Figure 1

Question 2:

Formulation of the Question: Can we predict the year a game was released using its estimated number of owners, price, and average playtime?

This is interesting because it allows us to explore whether player engagement and market size vary significantly over time, and if those patterns can help infer when a game was launched.

Design:

We used the cleaned Steam dataset from Kaggle. The features used for prediction were:

- estimated_owners (converted from a string range into a numeric average)
- price
- average_playtime_forever

The target variable was the release year, extracted from the release_date column. This was framed as a regression problem, and we applied two models: Linear Regression and K-Nearest Neighbors (KNN) Regression. We evaluated model performance using RMSE (Root Mean Squared Error) as our evaluation metric because it penalizes larger errors more heavily, making it ideal for identifying models that avoid big mistakes.

To prepare the dataset for modeling, we performed the following key preprocessing steps specific to this question:

- Extracted the year portion from the release_date column by slicing the first four characters of the date string.
- Converted the estimated_owners column from a string format to a numeric value by computing the average of the lower and upper bounds.
- Dropped any rows with missing values in the required columns.

The dataset was split into 80% training and 20% testing using a train-test split.

Implementation:

We treated the release year as a continuous numeric variable for regression. After scaling the features, we built regression pipelines for both Linear Regression and KNN.

For KNN, we used GridSearchCV to search over combinations of `n_neighbors` (1–20) and distance metrics (euclidean, manhattan). We used 5-fold cross-validation to find the best model configuration.

Results:

We evaluated both models using RMSE on the test set. The results were:

- Linear Regression RMSE: 3.532
- KNN Regression RMSE: 2.767

The best KNN model (using Euclidean distance with $k = 19$) performed significantly better than linear regression. This implies that the relationship between the game's features and its release year is nonlinear, and KNN's local distance-based approach was better able to capture those complexities.

```
Linear Regression RMSE: 3.532
KNN root mean squared error: 2.767
```

```
{'kneighborsregressor__metric': 'euclidean',
 'kneighborsregressor__n_neighbors': 19}
```

These results suggest that features like user engagement (playtime) and game popularity (estimated owners) have patterns that are historically relevant, but are not easily captured by a single linear model. KNN's lower error makes it a more effective model for predicting release year based on these behavioral and pricing features. (See Figure 2)

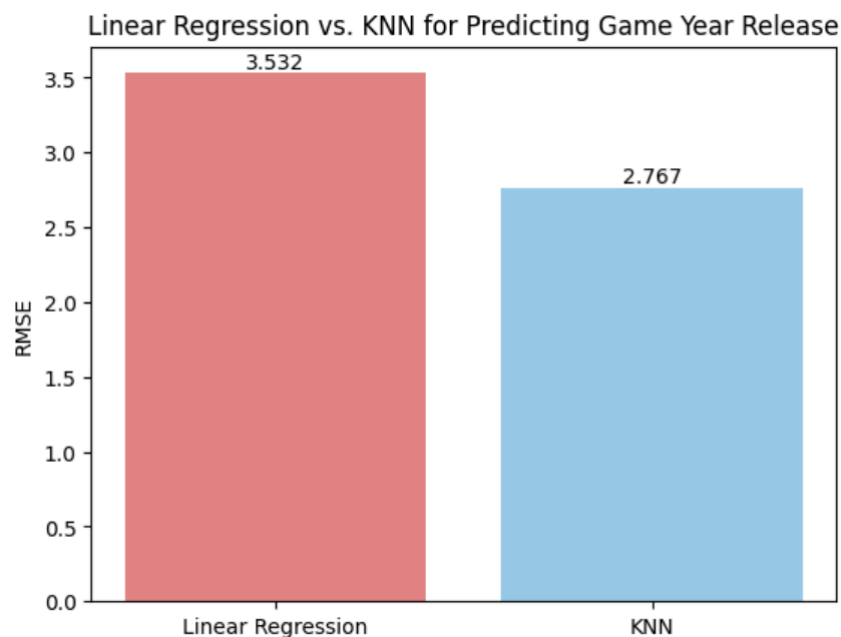


Figure 2

Question 3:

Formulation of the Question: Can we predict the number of concurrent users (CCU) a game will have based on its features?

The goal of this question is to explore whether game-specific features such as price, review counts, genre, and playtime can be used to estimate the peak number of concurrent users (CCU) a game experiences. This question is interesting because CCU is a key indicator of a game's popularity and real-time engagement.

Design:

We used the cleaned Steam dataset from Kaggle. The features selected for this task were:

- release_year, release_month, release_dayofweek (extracted from release_date)
- price, genres, positive, negative, num_reviews_total, estimated_owners, average_playtime_forever, and average_playtime_2weeks

The target variable was peak_ccu. We framed this as a regression problem and applied Linear Regression and K-Nearest Neighbors (KNN) Regression. Evaluation metrics included both Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) to capture prediction accuracy and average deviation.

To prepare the dataset:

- We extracted the year, month, and day of the week from the release_date column
- The genres column was transformed using one-hot encoding
- The estimated_owners field, which appeared as a string range (e.g., '20000-50000'), was converted to a single numeric value by taking the mean of the lower and upper bounds.

The data was split into 80% training and 20% testing using a train-test split.

Implementation:

Two pipelines were created:

- A Linear Regression pipeline that included scaling and encoding.
- A KNN Regressor pipeline, where we used GridSearchCV to tune hyperparameters.

The grid search tuned:

- n_neighbors: from 1 to 10
- metric: 'euclidean', 'manhattan'

We used 3-fold cross-validation (cv = 3) and scored models using negative mean squared error to find the best KNN configuration.

Results:

The KNN model significantly outperformed Linear Regression in predicting peak concurrent users.

Linear Regression:

- RMSE: 1046.07
- MAE: 210.59

KNN Regression (best model: Manhattan distance, 2 neighbors):

- RMSE: 532.65

- MAE: 33.97

```
print(f"RMSE: {lr_rmse}")
print(f"MAE: {lr_mae}")

RMSE: 1046.0674524652384
MAE: 210.58693984755809
```

```
Best Params: {'kneighborsregressor__metric': 'manhattan', 'kneighborsregressor__n_neighbors': 2}
RMSE: 532.6482715990422
MAE: 33.96914751171614
```

These results suggest that the KNN model, especially with a low number of neighbors and Manhattan distance, was more effective at capturing the localized, nonlinear patterns present in the data that influence CCU. The Linear Regression model struggled with the complexity of the relationships between features like user engagement, price, and ownership volume.

We visualized this performance comparison using a bar chart to highlight the improvement in accuracy achieved with KNN. (See Figure 3)

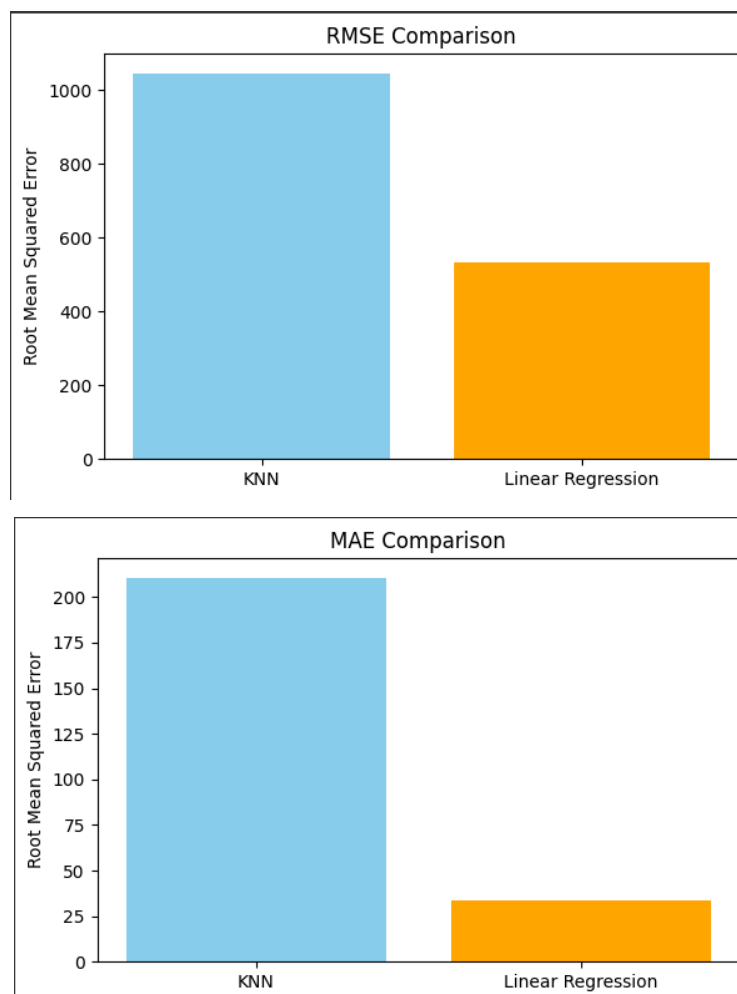


Figure 3

Question 4:

Formulation of the Question: Can we predict the total number of owners a game has based on its reviews, price, and genre?

This question is interesting because the number of game owners reflects overall popularity and market reach. Understanding which features are most correlated with ownership can help developers and marketers optimize game performance.

Design:

We used the cleaned Steam dataset and focused on the following features to predict `estimated_owners`:

- genres (categorical)
- peak_ccu, negative, positive (numerical review metrics)
- windows, mac, linux (boolean platform support)

The `estimated_owners` column was originally stored as a string range. We converted this into a single numeric value by taking the average of the lower and upper bounds. To speed up training and processing, we limited the dataset to the top 50 games based on review count.

The dataset was split into 80% training and 20% testing using a train-test split. We applied both Linear Regression and K-Nearest Neighbors Regression, using RMSE and MAE as evaluation metrics.

Implementation:

The most critical data transformation was converting `estimated_owners` from a range string into a numerical average. Boolean features such as windows, mac, and linux were converted from True/False to 1/0.

We built a KNN pipeline with GridSearchCV, tuning over different `n_neighbors` values and distance metrics (euclidean, manhattan) with 3-fold cross-validation. The best KNN model used Euclidean distance and 3 neighbors.

Results:

Model performance results were as follows:

Linear Regression

- RMSE: 1,039,450,657,160,485
- MAE: 27,309,176

KNN Regression (best: euclidean, k = 3)

- RMSE: 480,277,777,777,778
- MAE: 15,833,333

```
LR RMSE: 1039450657160485
LR MAE: 27309176
```

```
Best parameters found:
{'kneighborsregressor__metric': 'euclidean', 'kneighborsregressor__n_neighbors': 3}
KNN RMSE: 480277777777778
KNN MAE: 15833333
```

While the KNN model reduced the error compared to Linear Regression, both models still resulted in extremely high prediction errors. This indicates that features like price, platform

support, and reviews are not strongly correlated with the number of game owners. Game popularity likely depends on other factors not captured in the dataset, such as marketing, franchise reputation, or external trends, making ownership prediction highly challenging using just the available features. (See Figure 4)

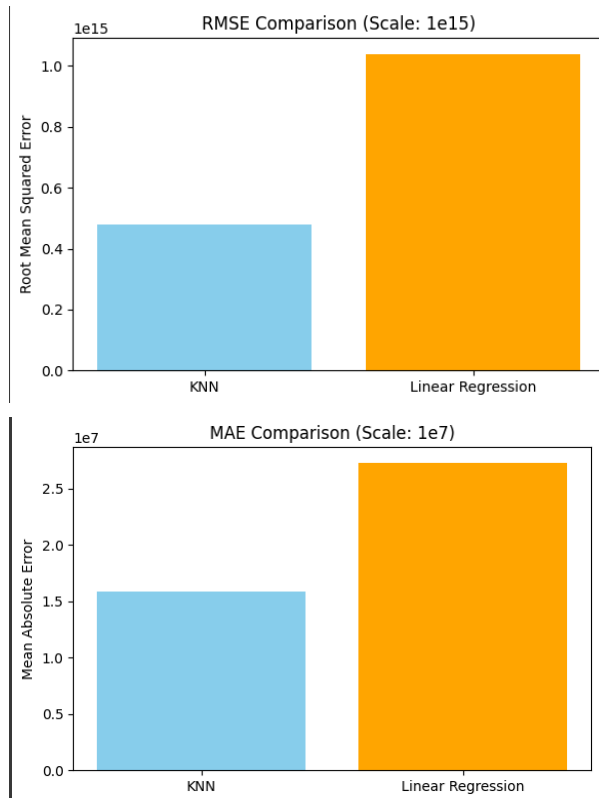


Figure 4

Question 5:

Formulation of the Question: Can we predict the number of reviews for a game based on its price and genre?

We chose this question to explore whether market-facing features, such as how much a game costs and what type of game it is, correlate with player engagement as measured by reviews.

Design:

For this task, we used the price and genres columns as input features, and the num_reviews_total column as the target variable. Since the genres column was stored as a stringified list (e.g., "[Action', 'Adventure']"), we needed to convert it into a usable format.

We applied Python's `ast.literal_eval()` to safely convert each genre string into a proper Python list. This was required in order to use `MultiLabelBinarizer()` for one-hot encoding genres across multiple categories.

After processing both features, we combined them into a single X_new DataFrame and used this as input for a K-Nearest Neighbors (KNN) Regressor. We evaluated the model using Root Mean Squared Error (RMSE), testing k-values from 1 to 14 using 5-fold cross-validation. The dataset was split into 80% training and 20% testing using train-test-split.

Implementation:

We looped through k-values from 1 to 14 and used cross_val_score with 5-fold cross-validation and neg_mean_squared_error to compute the RMSE for each k.

After identifying the best k based on the lowest average RMSE, we trained a final KNN model on the training set and we evaluated model performance using RMSE (Root Mean Squared Error) as our evaluation metric because it penalizes larger errors more heavily, making it ideal for identifying models that avoid big mistakes.

Results:

The best-performing KNN model used k = 12, achieving a Root Mean Squared Error (RMSE) of approximately 21,779 on the test set.

```
best k = 12
```

```
[ ] print(f"root mean squared error of best KNN model: {rmse}")
```

```
root mean squared error of best KNN model: 21779.734
```

This indicates that while genre and price do provide some predictive signal for the number of reviews, the relationship is still fairly noisy. The moderate RMSE suggests that user engagement (as measured by reviews) is influenced by additional factors beyond just price and genre, such as marketing, brand recognition, release timing, or social influence that aren't captured in this model.

Below is the visualization showing RMSE values across different k-values used to select the optimal number of neighbors. (See Figure 5)

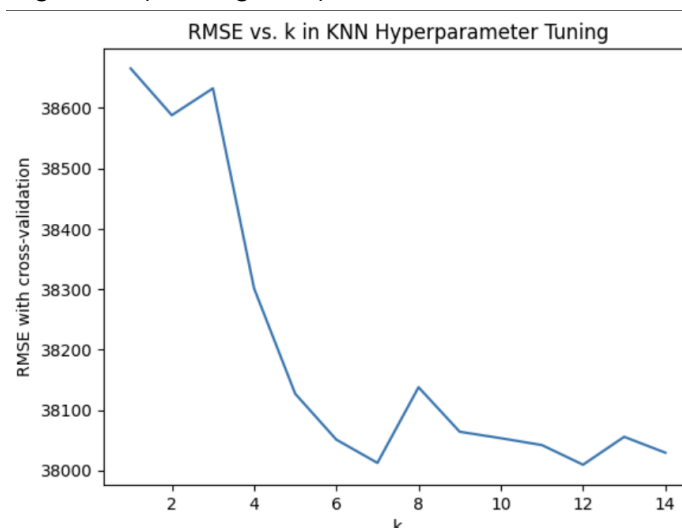


Figure 5

5.0 Discussion and Conclusions:

Across all five questions, our project focused on a central theme: using measurable game features to predict success and player engagement metrics. We explored how attributes like price, genre, playtime, reviews, and release timing influenced outcomes such as user sentiment, concurrent users, number of owners, and reviews. These are critical dimensions of how games perform in both commercial and user-experience terms.

A consistent takeaway was that K-Nearest Neighbors (KNN) outperformed linear models in most tasks, especially where relationships were complex, non-linear, or depended on local patterns. KNN delivered significantly better results for predicting review sentiment, concurrent user counts, and even release year. However, performance dropped for questions involving total ownership and review count, where available features didn't offer strong predictive signals. This suggests that success metrics like popularity are influenced by factors not captured in the dataset, such as game quality, marketing, or social media traction.

Overall, our models were able to uncover some useful trends, but also revealed the limitations of relying solely on structured gameplay and metadata. While internal features help capture patterns of engagement, integrating richer external data could substantially improve predictive power in future research.